1)WELCOME TO JAVA:

```
public class Solution {

    public static void main(String[] args) {
        /* Enter your code here. Print output to STDOUT. Your class should be named
Solution. */
        System.out.println("Hello, World.\nHello, Java.");

    }
}
```

---

2)Stdin and Stdout 1 :

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int a = scan.nextInt();
        int b = scan.nextInt();
        int c = scan.nextInt();

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);

    }
}
```

---

3)If-Else :

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;

public class Solution {



    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        int N = scanner.nextInt();

        scanner.close();
      if (N%2==1){
          System.out.println("Weird");
      }
      else if(N%2==0 && (N>=2 && N<=5)){
          System.out.println("Not Weird");
```

```
      }
      else if(N%2==0 && (N>5 && N<=20)){
          System.out.println("Weird");
      }
      else if(N%2==0 && (N>20 && N<=100)){
          System.out.println("Not Weird");
      }


    }
}
```

---

4)Stdin and Stdout 2 :

```
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int i = scanner.nextInt();
        double d = scanner.nextDouble();
        scanner.nextLine();
        String s = scanner.nextLine();



        System.out.println("String: " + s);
        System.out.println("Double: " + d);
        System.out.println("Int: " + i);
    }
}
```

---

5)Output Formatting :

```
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
            Scanner sc=new Scanner(System.in);
            System.out.println("================================");
            for(int i=0;i<3;i++){
                String s1=sc.next();
                int x=sc.nextInt();

                System.out.printf("%-15s%03d\n", s1, x);
            }
            System.out.println("================================");

    }
}
```

---

6)Java Loops 1 :

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;

public class Solution {

    private static final Scanner scanner = new Scanner(System.in);
    public static void main(String[] args) {
        int N = scanner.nextInt();
        scanner.skip("(\r\n|[\n\r\u2028\u2029\u0085])?");
        if(N>1 && N<=20)
        {
        for(int i=1 ; i<=10 ; i++)
          {
           int mul=N*i;
           System.out.println(N+" x "+i+" = "+mul);
           }
        }
        scanner.close();
    }
}
```

---

7) Java Loops 2 :

```
import java.util.*;
import java.io.*;

class Solution {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int queries = in.nextInt();

        for (int q = 0; q < queries; q++) {
            int a = in.nextInt();
            int b = in.nextInt();
            int n = in.nextInt();

            int result = a;

            for (int j = 0; j < n; j++) {
                result += Math.pow(2, j) * b;
                System.out.print(result + " ");
            }

            System.out.println();
        }

        in.close();
    }
}
```

---

8)DATATYPES:

```
import java.util.*;
import java.io.*;

class Solution{
    public static void main(String []argh)
    {
        Scanner sc = new Scanner(System.in);
        int t=sc.nextInt();
        for(int i=0;i<t;i++)
        {
            try
            {
                long x=sc.nextLong();
                System.out.println(x+" can be fitted in:");
                if(x>=-128 && x<=127)System.out.println("* byte");
                 if (x>=-32768 && x<=32767)System.out.println("* short");
                 if (x>=-2147483648 && x<=2147483647)System.out.println("* int");
                 if (x>=-9223372036854775808L &&
x<=9223372036854775807L)System.out.println("* long");
            }
            catch(Exception e)
            {
                System.out.println(sc.next()+" can't be fitted anywhere.");
            }

        }
    }
}
```

---

9)End-of-file:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT.
Your class should be named Solution. */
              Scanner sc = new Scanner(System.in);
               int c =1;
               while(sc.hasNext()){
               String s = sc.nextLine();
               System.out.println(c+" "+s);
               c++;
            }
        }
   }
```

---

10)Java Static Initializer Block:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);

        try{
            int b=sc.nextInt();
            int h=sc.nextInt();
            if (b <= 0 || h <= 0) {
                throw new Exception("java.lang.Exception: Breadth and height must
be positive");

            }
            System.out.println(b * h);

            }
    catch (Exception e){
            System.out.println(e.getMessage());

        }
        sc.close();
    }
}
```

---

11)Int to string:

to convert int to string method: String s= String.valueOf(n);

```
  try {
   Scanner in = new Scanner(System.in);
   int n = in .nextInt();
   in.close();
   //String s=???; Complete this line below
   String s= String.valueOf(n);


   if (n == Integer.parseInt(s)) {
    System.out.println("Good job");
   } else {
    System.out.println("Wrong answer.");
   }
  } catch (DoNotTerminate.ExitTrappedException e) {
   System.out.println("Unsuccessful Termination!!");
  }
```

---

12)Date and Time:

```
import java.time.LocalDate;

class Result {


    public static String findDay(int month, int day, int year) {
      LocalDate date = LocalDate.of(year, month, day);

        // Get the day of the week in capital letters
        String dayOfWeek = date.getDayOfWeek().toString();

        return dayOfWeek;


    }

}
```

---

13)Currency Formatter :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double payment = scanner.nextDouble();
        scanner.close();
        //WRITE CODE HERE
         String us = NumberFormat.getCurrencyInstance(Locale.US).format(payment);
         String india = NumberFormat.getCurrencyInstance(new
Locale("en","in")).format(payment);
          String china =
NumberFormat.getCurrencyInstance(Locale.CHINA).format(payment);
           String france =
NumberFormat.getCurrencyInstance(Locale.FRANCE).format(payment);

        // Print the formatted values
        System.out.println("US: " + us);
        System.out.println("India: " + india);
        System.out.println("China: " + china);
        System.out.println("France: " + france);
    }
}
```

(OR)

```
        // Create NumberFormat instances for different locales
        NumberFormat usFormat = NumberFormat.getCurrencyInstance(Locale.US);
        NumberFormat indiaFormat = NumberFormat.getCurrencyInstance(new
```

```
Locale("en", "IN"));
        NumberFormat chinaFormat = NumberFormat.getCurrencyInstance(Locale.CHINA);
        NumberFormat franceFormat =
NumberFormat.getCurrencyInstance(Locale.FRANCE);

        // Format the payment amount for each locale
        String us = usFormat.format(payment);
        String india = indiaFormat.format(payment);
        String china = chinaFormat.format(payment);
        String france = franceFormat.format(payment);
```

---

14)String introduction:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        String A=sc.next();
        String B=sc.next();
        /* Enter your code here. Print output to STDOUT. */

        System.out.println(A.length()+B.length());

        int ch = A.compareTo(B);

        if(ch < 0){
            System.out.println("No");
        }else if(ch == 0){
         System.out.println("No");

        }else{
                    System.out.println("Yes");

        }

        System.out.println(A.substring(0,1).toUpperCase()+A.substring(1)+"
"+B.substring(0,1).toUpperCase()+B.substring(1));

    }
}
```

---

15) substring:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
```

```
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        String S = in.next();
        int start = in.nextInt();
        int end = in.nextInt();
        System.out.println(S.substring(start,end));
        in.close();
    }
}
```

---

16) Substring Comparisons :

```
import java.util.Scanner;

public class Solution {

    public static String getSmallestAndLargest(String s, int k) {
        String smallest = "";
        String largest = "";

        // Complete the function

        // 'smallest' must be the lexicographically smallest substring of length'k'
        smallest = s.substring(0, k);
        largest = s.substring(0, k);

        // Iterate through the string to find smallest and largest substrings
        for (int i = 1; i <= s.length() - k; i++) {
            String substring = s.substring(i, i + k);

            // Compare the substring with the current smallest and largest
            if (substring.compareTo(smallest) < 0) {
                smallest = substring;
            } else if (substring.compareTo(largest) > 0) {
                largest = substring;
            }
        }

        // 'largest' must be the lexicographically largest substring of length 'k'

        return smallest + "\n" + largest;
    }


    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String s = scan.next();
        int k = scan.nextInt();
        scan.close();

        System.out.println(getSmallestAndLargest(s, k));
    }
}
```

---

17) String reverse:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        String A=sc.next();

        String rev="";
        for(int i=A.length()-1;i>=0;i--)
        {
            rev = rev+A.charAt(i);
        }
        if(rev.equals(A))
         System.out.print("Yes");
         else
         System.out.print("No");
        }
    }
```

---

18) Anagrams:

```
import java.util.Scanner;

public class Solution {

    static boolean isAnagram(String a, String b) {
        // Complete the function
                String s1 = a;
        String s2 = b;
        s1=s1.toLowerCase();
        s2=s2.toLowerCase();

        if(s1.length()==s2.length())
        {
            int[] arr = new int[256];
            int[] brr = new int[256];
            for (int i = 0; i < s1.length(); i++) {
                arr[(int) s1.charAt(i)] += 1;
                brr[(int) s2.charAt(i)] += 1;
            }
            for (int i = 0; i < 256; i++) {
                if (arr[i] != brr[i])
                    return false;
            }
            return true;
        }
        else
        {
            return false;
        }
    }
```

```
  public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        String a = scan.next();
        String b = scan.next();
        scan.close();
        boolean ret = isAnagram(a, b);
        System.out.println( (ret) ? "Anagrams" : "Not Anagrams" );
    }
}
```

---

19) String Tokens :

```
import java.io.*;
import java.util.*;
public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String s = scan.nextLine();
        scan.close();

        // Write your code here.
        s = s.trim();
        if (s.length() == 0) {
            System.out.println(0);
        } else {
            String[] strings = s.split("['!?,._@ ]+");
            System.out.println(strings.length);
            for (String str : strings)
                System.out.println(str);
        }
    }
}
```

---

20) Pattern Syntax Checker :

```
import java.util.Scanner;
import java.util.regex.*;

public class Solution
{
   public static void main(String[] args){
      Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());

        while(testCases>0){
            String pattern = in.nextLine();

            try {
                Pattern.compile(pattern);
                System.out.println("Valid");
            } catch(PatternSyntaxException e) {
                System.out.println("Invalid");
```

```
            }

            testCases--;
        }
   }
}
```

---

21) Java Regex :

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import java.util.Scanner;

class Solution{

    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        while(in.hasNext()){
            String IP = in.next();
            System.out.println(IP.matches(new MyRegex().pattern));
        }

    }
}

//Write your code here
class MyRegex
{
    String num = "([01]?\\d{1,2}|2[0-4]\\d|25[0-5])";
    String pattern = num + "." +  num + "." +  num + "." + num;
}
```

---

22) Java Regex 2 :

```
import java.util.Scanner;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class DuplicateWords {

    public static void main(String[] args) {

        String regex = "\\b(\\w+)(?:\\W+\\1\\b)+";
        Pattern p = Pattern.compile(regex,Pattern.CASE_INSENSITIVE);

        Scanner in = new Scanner(System.in);
        int numSentences = Integer.parseInt(in.nextLine());

        while (numSentences-- > 0) {
            String input = in.nextLine();

            Matcher m = p.matcher(input);
```

```
            // Check for subsequences of input that match the compiled pattern
            while (m.find()) {
                input = input.replaceAll(m.group(), m.group(1));
            }

            // Prints the modified sentence.
            System.out.println(input);
        }

        in.close();
    }
}
```

---

23) Valid username regular expression : "^[A-Za-z]\\w{7,29}$"

```
import java.util.Scanner;
class UsernameValidator {
    /*
     * Write regular expression here.
     */
    public static final String regularExpression = "^[A-Za-z]\\w{7,29}$";
}


public class Solution {
    private static final Scanner scan = new Scanner(System.in);

    public static void main(String[] args) {
        int n = Integer.parseInt(scan.nextLine());
        while (n-- != 0) {
            String userName = scan.nextLine();

            if (userName.matches(UsernameValidator.regularExpression)) {
                System.out.println("Valid");
            } else {
                System.out.println("Invalid");
            }
        }
    }
}
```

---

24) Tag Content Extractor :

---

25) Big Decimal :

---

26) Primalirity test :

```
import java.io.*;
import java.math.*;
```

```
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        String n = bufferedReader.readLine();
        BigInteger bi = new BigInteger(n);
        System.out.println(bi.isProbablePrime(10) ? "prime" : "not prime");
        bufferedReader.close();
    }
}
```

---

27) Big INteger :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BigInteger big1 = new BigInteger(sc.nextLine());
        BigInteger big2 = new BigInteger(sc.nextLine());

        BigInteger sum, product;

        sum = big1.add(big2);
        product = big1.multiply(big2);

        System.out.println(sum+"\n"+product);

    }
}
```

---

28) 1D array :

```
import java.util.*;

public class Solution {
```

```
    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
         int[] a = new int[n];


        for(int i=0;i<n;i++)
     {
          a[i]=scan.nextInt();
     }

        scan.close();

        // Prints each sequential element in array a
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

---

29) 2D array :

---

30) Subarray :

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT.
Your class should be named Solution. */
                int n,sum=0,c=0;
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        int[] a = new int[n];
        for(int i=0;i<n;i++){
            a[i] = sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            for(int j=i;j<n;j++)
            {
                sum = 0;
                for(int k=i;k<=j;k++)
                {
                    sum+=a[k];

                }
                 if(sum<0)c++;
            }
```

```
        }
        System.out.print(c);

    }
}
```

---

31) ArrayList :

---

32) 1D array (part 2) :

---

33) List :

```
import java.util.Scanner;
import java.util.LinkedList;

public class Solution {
    public static void main(String[] args) {
        /* Create and fill Linked List of Integers */
        Scanner scan = new Scanner(System.in);
        int N = scan.nextInt();
        LinkedList<Integer> list = new LinkedList<>();
        for (int i = 0; i < N; i++) {
            int value = scan.nextInt();
            list.add(value);
        }

        /* Perfrom queries on Linked List */
        int Q = scan.nextInt();
        for (int i = 0; i < Q; i++) {
            String action = scan.next();
            if (action.equals("Insert")) {
                int index = scan.nextInt();
                int value = scan.nextInt();
                list.add(index, value);
            } else { // "Delete"
                int index = scan.nextInt();
                list.remove(index);
            }
        }
        scan.close();

        /* Print our updated Linked List */
        for (Integer num : list) {
            System.out.print(num + " ");
        }
    }
}
```

Inheritance 1 :

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

class Animal{
      void walk()
      {
            System.out.println("I am walking");
      }
}
class Bird extends Animal
{
      void fly()
      {
            System.out.println("I am flying");
      }
    void sing()
    {
        System.out.println("I am singing");
    }
}

public class Solution{

   public static void main(String args[]){

        Bird bird = new Bird();
        bird.walk();
        bird.fly();
      bird.sing();

   }
}
```

Inheritance 2 :

```
class Arithmetic {
    int add(int n1,int n2) {
        return n1+n2;

    }
}

class Adder extends Arithmetic {

    }



public class Solution{
    public static void main(String []args){
        // Create a new Adder object
        Adder a = new Adder();
```

```
        // Print the name of the superclass on a new line
        System.out.println("My superclass is: " +
a.getClass().getSuperclass().getName());

        // Print the result of 3 calls to Adder's `add(int,int)` method as 3 space-
separated integers:
        System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) + "\
n");
     }
}
```

Exception Handling (Try Catch) :

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT.
Your class should be named Solution. */
        try{
            Scanner sc = new Scanner(System.in);
            int a = sc.nextInt();

            int b = sc.nextInt();

            int c = a/b;
            System.out.print(c);
        }
        catch(InputMismatchException ob){
            System.out.print("java.util.InputMismatchException");
        }
        catch(Exception e)
        {
            System.out.print(e);
        }
    }
}
```

Big Decimal :

```
import java.math.BigDecimal;
import java.util.*;
class Solution{
    public static void main(String []args){
        //Input
        Scanner sc= new Scanner(System.in);
        int n=sc.nextInt();
        String []s=new String[n+2];
        for(int i=0;i<n;i++){
            s[i]=sc.next();
        }
```

```
        sc.close();

        //Write your code here

        for(int i=0;i<n;i++)
{
    //inserting string values to bigdecimal
    BigDecimal First=new BigDecimal(s[i]);
    int index=i;
    for(int j=i+1;j<n;j++)
    {
        //second BigDecimal to compare the first Bigdecimal
        BigDecimal Second=new BigDecimal(s[j]);
        //comparing if First element is greater that second element
        //if the First element is greater than Second element than compareTo()
returns 1
        if(Second.compareTo(First)==1){
            First=Second;
            index=j;
        }
    }
    //temporary variable to store s[i] value
        String temp=s[i];
        s[i]=s[index];
        s[index]=temp;
}

        //Output
        for(int i=0;i<n;i++)
        {
            System.out.println(s[i]);
        }
    }
}

----------------------

Exception Handling :

import java.util.Scanner;
import java.util.Scanner;
import java.math.*;
class MyCalculator {
    /*
    * Create the method long power(int, int) here.
    */

    public static long power(int a,int b) throws Exception{
        long la = a;
        long lb = b;
             long c = (long)Math.pow(la,lb);
             if(la==0 && lb==0)  {
                 throw new Exception("n and p should not be zero.");

             }
             else if(la<0 || lb <0) {
             throw new Exception("n or p should not be negative.");

             }
```

```
            else
            return c;

    }


}

public class Solution {
    public static final MyCalculator my_calculator = new MyCalculator();
    public static final Scanner in = new Scanner(System.in);

    public static void main(String[] args) {
        while (in .hasNextInt()) {
            int n = in .nextInt();
            int p = in .nextInt();

            try {
                System.out.println(my_calculator.power(n, p));
            } catch (Exception e) {
                System.out.println(e);
            }
        }
    }
}


-----------------------

Method Overriding :

import java.util.*;
class Sports{

    String getName(){
        return "Generic Sports";
    }

    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}

class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }


    // Write your overridden getNumberOfTeamMembers method here
        @Override
    void getNumberOfTeamMembers() {
        System.out.println( "Each team has 11 players in " + getName() );


    }

}
```

```
public class Solution{

    public static void main(String []args){
        Sports c1 = new Sports();
        Soccer c2 = new Soccer();
        System.out.println(c1.getName());
        c1.getNumberOfTeamMembers();
        System.out.println(c2.getName());
        c2.getNumberOfTeamMembers();
      }
}
```

Method Overriding (Super keyword) : String temp=super.define_me();

```
import java.util.*;
import java.io.*;


class BiCycle{
      String define_me(){
            return "a vehicle with pedals.";
      }
}

class MotorCycle extends BiCycle{
      String define_me(){
            return "a cycle with an engine.";
      }

      MotorCycle(){
            System.out.println("Hello I am a motorcycle, I am "+ define_me());

            String temp=super.define_me(); //Fix this line

            System.out.println("My ancestor is a cycle who is "+ temp );
      }

}
class Solution{
      public static void main(String []args){
            MotorCycle M=new MotorCycle();
      }
}
```

Factory Pattern :

```
import java.util.*;
import java.security.*;
interface Food {
       public String getType();
      }
      class Pizza implements Food {
       public String getType() {
```

```
       return "Someone ordered a Fast Food!";
       }
      }

      class Cake implements Food {

       public String getType() {
       return "Someone ordered a Dessert!";
       }
      }
      class FoodFactory {
            public Food getFood(String order) {

switch (order){
     case "pizza": return new Pizza();
     case "cake" : return new Cake();
     default : return null;
}

}//End of getFood method

      }//End of factory class

      public class Solution {

       public static void main(String args[]){
                  Do_Not_Terminate.forbidExit();

            try{

                  Scanner sc=new Scanner(System.in);
                  //creating the factory
                  FoodFactory foodFactory = new FoodFactory();

                  //factory instantiates an object
                  Food food = foodFactory.getFood(sc.nextLine());


                  System.out.println("The factory returned "+food.getClass());
                  System.out.println(food.getType());
            }
            catch (Do_Not_Terminate.ExitTrappedException e) {
                  System.out.println("Unsuccessful Termination!!");
            }
       }

      }
      class Do_Not_Terminate {

          public static class ExitTrappedException extends SecurityException {

                  private static final long serialVersionUID = 1L;
          }

          public static void forbidExit() {
              final SecurityManager securityManager = new SecurityManager() {
                  @Override
                  public void checkPermission(Permission permission) {
                      if (permission.getName().contains("exitVM")) {
```

```
                        throw new ExitTrappedException();
                    }
                }
            };
            System.setSecurityManager(securityManager);
        }
    }
```

Singleton Pattern :

```
class Singleton{

        static Singleton instance = null;
        public String str ;

    private Singleton(){

    }
    static public Singleton getSingleInstance()
    {
        if (instance == null)
             instance = new Singleton();

        return instance;
    }
}

class Solution{
     public static void main(String args[]){
     Singleton a = Singleton.getSingleInstance ();

  }
}
```

InstanceOf() :

```
import java.util.*;


class Student{}
class Rockstar{   }
class Hacker{}


public class InstanceOFTutorial{

   static String count(ArrayList mylist){
      int a = 0,b = 0,c = 0;
      for(int i = 0; i < mylist.size(); i++){
         Object element=mylist.get(i);
     if(element instanceof Student )
        a++;
     if(element instanceof Rockstar)
        b++;
     if(element instanceof Hacker)
        c++;
```

```
      }
      String ret = Integer.toString(a)+" "+ Integer.toString(b)+" "+
Integer.toString(c);
      return ret;
   }

   public static void main(String []args){
      ArrayList mylist = new ArrayList();
      Scanner sc = new Scanner(System.in);
      int t = sc.nextInt();
      for(int i=0; i<t; i++){
         String s=sc.next();
         if(s.equals("Student"))mylist.add(new Student());
         if(s.equals("Rockstar"))mylist.add(new Rockstar());
         if(s.equals("Hacker"))mylist.add(new Hacker());
      }
      System.out.println(count(mylist));
   }
}
```

Java Interface :

```
import java.util.*;
interface AdvancedArithmetic{
  int divisor_sum(int n);
}

//Write your code here
class MyCalculator implements AdvancedArithmetic {
    public int divisor_sum(int n) {
        int sum=0;
        for(int i=1;i<=n;i++) {
            if(n%i==0)
                sum+=i;
        }
        return sum;
    }
}

class Solution{
    public static void main(String []args){
        MyCalculator my_calculator = new MyCalculator();
        System.out.print("I implemented: ");
        ImplementedInterfaceNames(my_calculator);
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        System.out.print(my_calculator.divisor_sum(n) + "\n");
            sc.close();
    }
    /*
     *  ImplementedInterfaceNames method takes an object and prints the name of the
interfaces it implemented
     */
    static void ImplementedInterfaceNames(Object o){
        Class[] theInterfaces = o.getClass().getInterfaces();
        for (int i = 0; i < theInterfaces.length; i++){
            String interfaceName = theInterfaces[i].getName();
```

```
            System.out.println(interfaceName);
        }
    }
}
```

---

java SORT :

```
import java.util.*;

class Student{
      private int id;
      private String fname;
      private double cgpa;
      public Student(int id, String fname, double cgpa) {
            super();
            this.id = id;
            this.fname = fname;
            this.cgpa = cgpa;
      }
      public int getId() {
            return id;
      }
      public String getFname() {
            return fname;
      }
      public double getCgpa() {
            return cgpa;
      }
}

//Complete the code
public class Solution
{
      public static void main(String[] args){
            Scanner in = new Scanner(System.in);
            int testCases = Integer.parseInt(in.nextLine());

            List<Student> studentList = new ArrayList<Student>();
            while(testCases>0){
                  int id = in.nextInt();
                  String fname = in.next();
                  double cgpa = in.nextDouble();

                  Student st = new Student(id, fname, cgpa);
                  studentList.add(st);

                  testCases--;
            }

         Collections.sort(studentList,  Comparator.comparing(Student ::
getCgpa).reversed().thenComparing(Student :: getFname).thenComparing(Student ::
getId));

            for(Student st: studentList){
                  System.out.println(st.getFname());
            }
      }
```

```
}


-------------


java abstract class :

import java.util.*;
abstract class Book{
      String title;
      abstract void setTitle(String s);
      String getTitle(){
            return title;
      }

}

//Write MyBook class here
class MyBook extends Book {
    @Override
    void setTitle(String s) {
        title = s;
    }
}

public class Main{

      public static void main(String []args){
            //Book new_novel=new Book(); This line prHMain.java:25: error: Book is
abstract; cannot be instantiated
            Scanner sc=new Scanner(System.in);
            String title=sc.nextLine();
            MyBook new_novel=new MyBook();
            new_novel.setTitle(title);
            System.out.println("The title is: "+new_novel.getTitle());
            sc.close();

      }
}
```